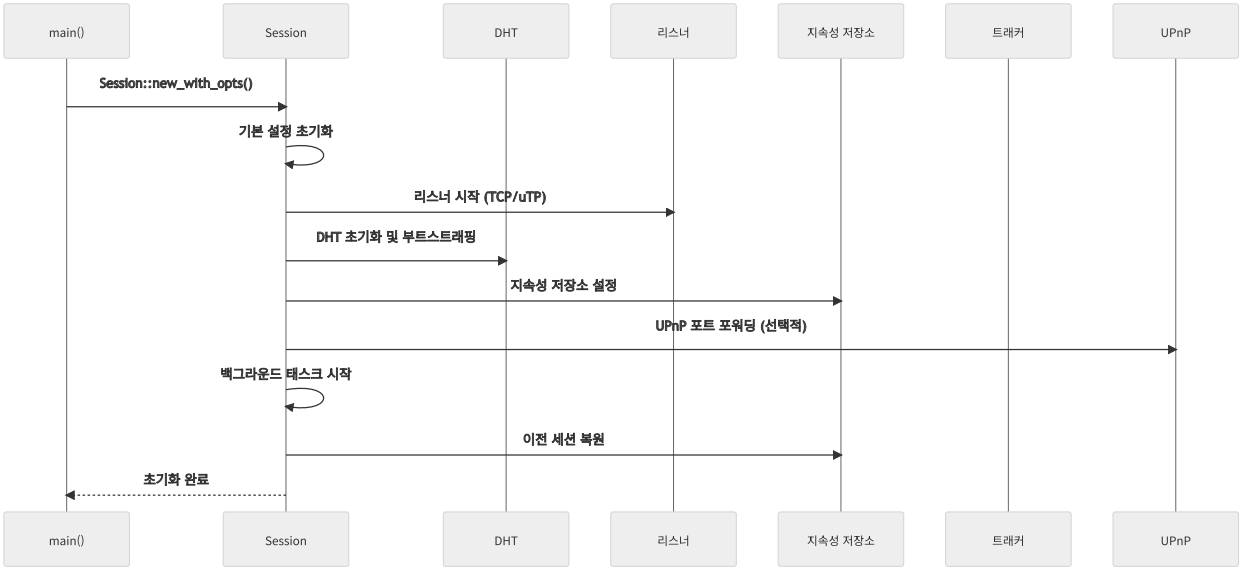
rqbit 시스템 시작 및 종료 과정 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 애플리케이션 시작 시 초기화 순서, 정상 종료와 비정상 종료 시의 정리 과정, 그리고 리소스 해제와 상태 저장 메커니즘을 상세히 분석합니다.

시스템 시작 과정

1. 애플리케이션 진입점



1.1 Session 생성 및 초기화

Session::new_with_opts 메서드:

```
pub  async  fn new_with_opts(mut opts: SessionOptions) -> anyhow::Result<Arc<Sel

// 1. 기본 설정 초기화

let peer_id = opts.peer_id.unwrap_or_else(||

generate_azereus_style(*b"rQ", crate_version!())
```

```
);
let token = opts.cancellation_token.take().unwrap_or_default();
let _cancellation_token_drop_guard = token.clone().drop_guard();


// 2. 네트워크 바인딩 설정
let bind_device = match opts.bind_device_name.as_ref() {
Some(name) => Some(BindDevice::new_from_name(name)?),
None => None,
};


// 3. 리스너 시작
let listen_result = if let Some(listen_opts) = opts.listen.take() {
Some(listen_opts.start(
token.clone(),
bind_device.as_ref(),
debug_span!(parent: None, "listen"),
).await?)
} else {
None
};


// 4. DHT 초기화
let dht = if opts.disable_dht {
```

```
None
} else {
Some(initialize_dht(&opts, &token, &bind_device).await?)
};


// 5. 지속성 저장소 설정
let (persistence, bitv_factory) = persistence_factory(&opts).await?;


// 6. 네트워크 클라이언트 초기화
let (reqwest_client, udp_tracker_client) = initialize_network_clients(&opts).awa


// 7. Session 구조체 생성
let session = Arc::new(Session {
db: RwLock::new(SessionDatabase::default()),
next_id: AtomicUsize::new(0),
bitv_factory,
spawner: BlockingSpawner::default(),
peer_id,
announce_port: listen_result.as_ref().and_then(|r| r.announce_port),
listen_addr: listen_result.as_ref().map(|r| r.addr),
dht,
connector: Arc::new(StreamConnector::new(/* ... */)),
reqwest_client,
```

```
udp_tracker_client,
// ... 기타 필드들
cancellation_token: token,
_cancellation_token_drop_guard,
});

// 8. 백그라운드 태스크 시작
session.start_background_tasks().await?;

// 9. 이전 세션 복원
session.restore_previous_session().await?;

Ok(session)
}
```

1.2 리스너 초기화 순서

ListenerOptions::start 메서드:

```
pub async fn start(
self,
cancellation_token: CancellationToken,
bind_device: Option<&BindDevice>,
span: tracing::Span,
```

```
) -> anyhow::Result<ListenResult> {
let mut addr = self.listen_addr;
let mut tcp_socket = None;
let mut utp_socket = None;


// 1. uTP 옵션 설정
let mut utp_opts = self.utp_opts.unwrap_or_default();
utp_opts.cancellation_token = Some(cancellation_token.clone());
utp_opts.parent_span = Some(span.clone());
utp_opts.wait_for_lastack = false;


// 2. TCP 리스너 시작 (활성화된 경우)
if self.mode.tcp_enabled() {
match start_tcp_listener(addr, bind_device).await {
Ok((listener, bound_addr)) => {
tcp_socket = Some(listener);
addr = bound_addr;
info!("Listening on TCP {:?} for incoming peer connections", addr);
}
Err(e) => {
error!("Failed to bind TCP listener: {:#}", e);
return Err(e);
}
```

```
}
}

// 3. uTP 리스너 시작 (활성화된 경우)
if self.mode.utp_enabled() {
match start_utp_listener(addr, utp_opts, bind_device).await {
Ok(socket) => {
utp_socket = Some(socket);
info!("Listening on UDP {:?} for incoming uTP peer connections", addr);
}
Err(e) => {
if tcp_socket.is_none() {
// TCP도 실패했으면 치명적 오류
error!("Failed to bind uTP listener and TCP is disabled: {:#}", e);
return Err(e);
} else {
// TCP가 성공했으면 경고만
warn!("Failed to bind uTP listener: {:#}", e);
}
}
}
}
```

```
Ok(ListenResult {
tcp_socket,
utp_socket,
enable_upnp_port_forwarding: self.enable_upnp_port_forwarding,
addr,
announce_port: if addr.ip().is_loopback() { None } else { Some(addr.port()) },
})
}
```

1.3 DHT 초기화 과정

DHT 초기화 단계:

```
async fn initialize_dht(
opts: &SessionOptions,
token: &CancellationToken,
bind_device: &Option<BindDevice>,
) -> anyhow::Result<Dht> {
// 1. DHT 설정 구성
let dht_config = DhtConfig {
cancellation_token: Some(token.clone()),
bind_device: bind_device.clone(),
disable_persistence: opts.disable_dht_persistence,
persistence_filename: opts.dht_persistence_filename.clone(),
```

```
};

// 2. DHT 인스턴스 생성
let dht = Dht::new(dht_config).await?;

// 3. 부트스트래핑 시작
if !opts.dht_bootstrap_nodes.is_empty() {
for bootstrap_node in &opts.dht_bootstrap_nodes {
dht.bootstrap_hostname_with_backoff(bootstrap_node).await?;
}
} else {
// 기본 부트스트래핑 노드들
dht.bootstrap_default_nodes().await?;
}

// 4. 지속성 데이터 로드
if !opts.disable_dht_persistence {
dht.load_persistence_data().await?;
}

Ok(dht)
}
```

1.4 백그라운드 태스크 시작

Session::start_background_tasks 메서드:

```
async fn start_background_tasks(&self) -> anyhow::Result<()> {
// 1. TCP 리스너 태스크
if let Some(tcp_listener) = &self.tcp_socket {
self.spawn(
debug_span!(parent: self.rs(), "tcp_listener"),
"tcp_listener",
{
let session = Arc::downgrade(self);
let listener = tcp_listener.clone();
async move {
accept_incoming_connections(session, listener, ConnectionKind::Tcp).await
}
},
);
}

// 2. uTP 리스너 태스크
if let Some(utp_socket) = &self.utp_socket {
self.spawn(
debug_span!(parent: self.rs(), "utp_listener"),
```

```
"utp_listener",
{
let session = Arc::downgrade(self);
let socket = utp_socket.clone();
async move {
accept_incoming_connections(session, socket, ConnectionKind::Utp).await
}
},
);
}


// 3. 디스크 쓰기 워커 (선택적)
if let Some(disk_write_tx) = &self.disk_write_tx {
self.spawn(
debug_span!(parent: self.rs(), "disk_writer"),
"disk_writer",
{
let mut rx = disk_write_tx.subscribe();
async move {
while let Some(work) = rx.recv().await {
spawner.spawn_block_in_place(work);
}
Ok(())
```

```
}
},
);
}

// 4. 속도 추정기 업데이터
self.start_speed_estimator_updater();

// 5. UPnP 포트 포워딩 (선택적)
if self.enable_upnp_port_forwarding {
self.start_upnp_port_forwarding().await?;
}

Ok(())
}
```

1.5 이전 세션 복원

Session::restore_previous_session 메서드:

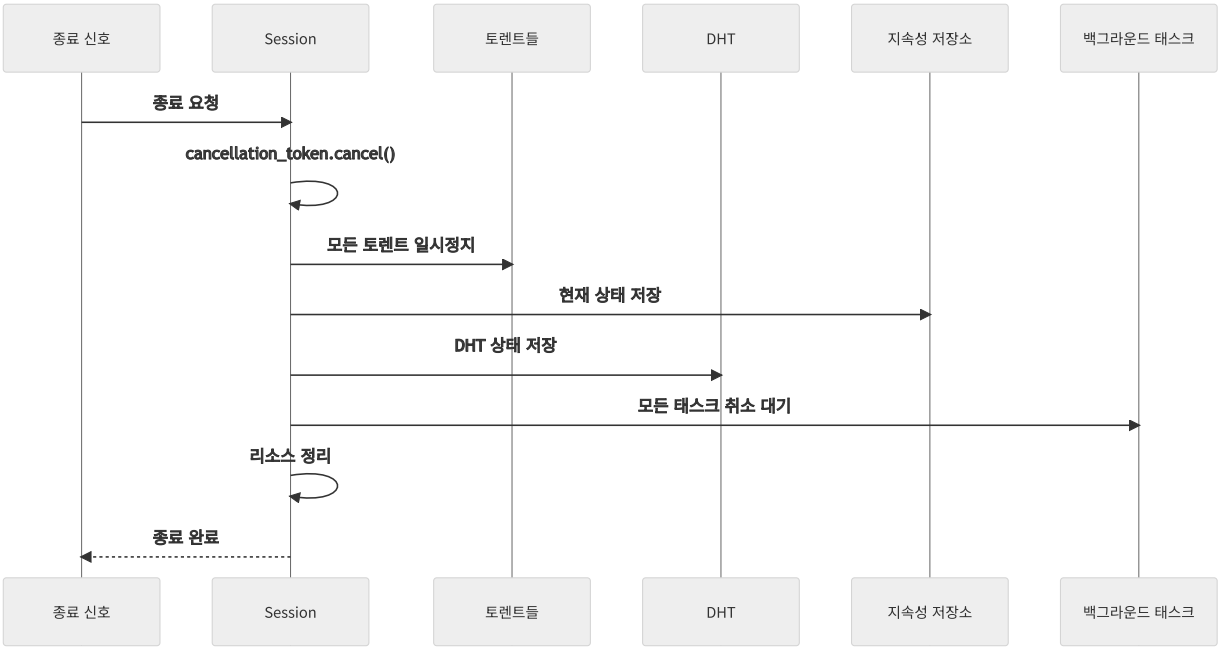
```
async fn restore_previous_session(&self) -> anyhow::Result<()> {
if let Some(persistence) = self.persistence.as_ref() {
info!("Restoring previous session from persistence store");
```

```
let mut ps = persistence.stream_all().await?;

let mut added_all = false;

let mut futs = FuturesUnordered::new();



// 병렬로 토렌트 복원

while !added_all || !futs.is_empty() {

tokio::select! {

// 복원 완료된 토렌트 처리

Some(res) = futs.next(), if !futs.is_empty() => {

match res {

Ok(response) => {

info!("Successfully restored torrent: {:?}", response);

}

Err(e) => {

error!("Error restoring torrent: {:#}", e);

}

}

}

// 새 토렌트 복원 시작

st = ps.next(), if !added_all => {

match st {

Some(st) => {

let (id, serialized_torrent) = st?;
```

```
let (add_torrent, mut opts) = serialized_torrent.into_add_torrent()?;

opts.preferred_id = Some(id);

let fut = self.add_torrent(add_torrent, Some(opts));

futs.push(fut);

},

None => {

added_all = true;

info!("Finished queuing all torrents for restoration");

}

};

}

};

}



info!("Session restoration completed");

}

Ok(())

}
```

시스템 종료 과정

2. 정상 종료 메커니즘



2.1 취소 토큰 기반 종료

CancellationToken 활용:

```
impl Session {
pub fn shutdown(&self) {
info!("Initiating graceful shutdown");
// 1. 모든 하위 태스크에 취소 신호 전송
self.cancellation_token.cancel();
```

```
// 2. 취소 완료 대기는 DropGuard에서 자동 처리
}
}

// DropGuard를 통한 자동 정리
impl Drop for Session {
fn drop(&mut self) {
// 취소 토큰이 자동으로 모든 태스크 정리
debug!("Session dropped, all background tasks will be cancelled");
}
}
```

2.2 토렌트별 정리 과정

ManagedTorrent 종료:

```
impl ManagedTorrent {
pub fn shutdown(&self) -> anyhow::Result<()> {
let mut g = self.locked.write();
match g.state.take() {
ManagedTorrentState::Live(live) => {
info!("Shutting down live torrent {}", self.shared.id);
// 1. 모든 피어 연결 정리
live.shutdown_all_peers()?;
```

```
// 2. 진행 상황 저장
if let Some(persistence) = &self.shared.session.upgrade()?.persistence {
persistence.store(self.shared.id, self)?;
}
// 3. 상태를 Paused로 전환
let paused = live.into_paused()?;
g.state = ManagedTorrentState::Paused(paused);
g.paused = true;
}
ManagedTorrentState::Initializing(init) => {
info!("Shutting down initializing torrent {}", self.shared.id);
// 초기화 중인 토렌트는 취소
init.cancel();
g.state = ManagedTorrentState::Error(anyhow!("Shutdown during initialization"));
}
ManagedTorrentState::Paused(_) => {
// 이미 일시정지 상태
debug!("Torrent {} already paused", self.shared.id);
}
ManagedTorrentState::Error(_) => {
// 오류 상태는 그대로 유지
debug!("Torrent {} in error state", self.shared.id);
}
```

```
}
self.state_change_notify.notify_waiters();
Ok(())
}
}
```

2.3 DHT 종료 과정

DHT 상태 저장 및 정리:

```
impl Dht {
pub async fn shutdown(&self) -> anyhow::Result<()> {
info!("Shutting down DHT");
// 1. 라우팅 테이블 저장
if let Some(persistence_file) = &self.persistence_filename {
self.save_routing_table(persistence_file).await?;
}
// 2. 진행 중인 쿼리들 취소
self.cancel_all_queries().await;
// 3. UDP 소켓 정리
self.worker_sender.send(WorkerCommand::Shutdown)?;
info!("DHT shutdown completed");
Ok(())
}
```

```rust
async fn save_routing_table(&self, filename: &Path) -> anyhow::Result<()> {
let nodes_v4: Vec<_> = self.routing_table_v4.read()
.iter()
.filter(|node| matches!(node.status(now()), NodeStatus::Good))
.map(|node| CompactNodeInfo::from(node))
.collect();
let nodes_v6: Vec<_> = self.routing_table_v6.read()
.iter()
.filter(|node| matches!(node.status(now()), NodeStatus::Good))
.map(|node| CompactNodeInfo::from(node))
.collect();
let data = DhtPersistenceData {
nodes_v4,
nodes_v6,
timestamp: SystemTime::now(),
};
// 원자적 쓰기
let tmp_file = format!("{}.tmp", filename.display());
let serialized = serde_json::to_vec_pretty(&data)?;
tokio::fs::write(&tmp_file, serialized).await?;
tokio::fs::rename(&tmp_file, filename).await?;
info!("Saved {} IPv4 and {} IPv6 nodes to DHT persistence",
nodes_v4.len(), nodes_v6.len());
```

```
Ok(())
}
}
```

2.4 지속성 저장소 플러시

SessionPersistenceStore 종료:

```
impl JsonSessionPersistenceStore {
pub async fn shutdown(&self) -> anyhow::Result<()> {
info!("Flushing session persistence store");
// 1. 모든 변경사항 디스크에 플러시
self.flush().await?;
// 2. 비트필드 파일들 플러시
for (id, _) in self.db_content.read().await.torrents.iter() {
if let Ok(Some(mut bitv)) = self.load((*id).into()).await {
if let Err(e) = bitv.flush() {
warn!("Failed to flush bitfield for torrent {}: {:#}", id, e);
}
}
}
info!("Session persistence store shutdown completed");
Ok(())
}
```

```rust
async fn flush(&self) -> anyhow::Result<()> {
let db_content = self.db_content.read().await;
let tmp_filename = format!("{}.tmp", self.db_filename.display());
// 원자적 쓰기
let mut tmp = tokio::fs::OpenOptions::new()
.create(true)
.truncate(true)
.write(true)
.open(&tmp_filename)
.await?;

let serialized = serde_json::to_vec_pretty(&*db_content)?;
tmp.write_all(&serialized).await?;
tmp.sync_all().await?;
drop(tmp);

tokio::fs::rename(&tmp_filename, &self.db_filename).await?;
Ok(())
}
}
```

비정상 종료 처리

3. 신호 처리 및 복구

신호 핸들러 설정:

```
#[cfg(unix)]
async fn setup_signal_handlers(session: Arc<Session>) -> anyhow::Result<()> {
use tokio::signal::unix::{signal, SignalKind};
let mut sigterm = signal(SignalKind::terminate())?;
let mut sigint = signal(SignalKind::interrupt())?;
tokio::spawn(async move {
tokio::select! {
_ = sigterm.recv() => {
info!("Received SIGTERM, initiating graceful shutdown");
session.shutdown();
}
_ = sigint.recv() => {
info!("Received SIGINT (Ctrl+C), initiating graceful shutdown");
session.shutdown();
}
}
});
Ok(())
}
```

```rust
#[cfg(windows)]
async fn setup_signal_handlers(session: Arc<Session>) -> anyhow::Result<()> {
use tokio::signal::windows::{ctrl_c, ctrl_break};
let mut ctrl_c = ctrl_c()?;
let mut ctrl_break = ctrl_break()?;
tokio::spawn(async move {
tokio::select! {
_ = ctrl_c.recv() => {
info!("Received Ctrl+C, initiating graceful shutdown");
session.shutdown();
}
_ = ctrl_break.recv() => {
info!("Received Ctrl+Break, initiating graceful shutdown");
session.shutdown();
}
}
});
Ok(())
}
```

3.1 패닉 핸들러

패닉 시 상태 저장:

```rust
fn setup_panic_handler(session: Arc<Session>) {
let original_hook = std::panic::take_hook();
std::panic::set_hook(Box::new(move |panic_info| {
error!("Application panicked: {}", panic_info);
// 긴급 상태 저장 시도
if let Some(persistence) = &session.persistence {
warn!("Attempting emergency state save due to panic");
// 블로킹 방식으로 즉시 저장
if let Err(e) = tokio::runtime::Handle::try_current()
.and_then(|handle| {
handle.block_on(async {
for (id, torrent) in session.db.read().torrents.iter() {
if let Err(e) = persistence.store(*id, torrent).await {
warn!("Failed to save torrent {} during panic: {:#}", id, e);
}
}
Ok::<_, anyhow::Error>(())
})
}) {
error!("Emergency state save failed: {:#}", e);
} else {
info!("Emergency state save completed");
```

```
}

}

// 원래 패닉 핸들러 호출

original_hook(panic_info);

}));

}
```

3.2 손상된 데이터 복구

FastResume 데이터 검증 및 복구:

```
async fn validate_and_recover_fastresume(

&self,

bitv_factory: &dyn BitVFactory,

have_pieces: Option<Box<dyn BitV>>,

) -> Option<Box<dyn BitV>> {

let hp = have_pieces?;

// 1. 기본 검증

let actual = hp.as_bytes().len();

let expected = self.metadata.lengths().piece_bitfield_bytes();

if actual != expected {

warn!("Bitfield length mismatch (expected: {}, actual: {}), performing full chec

expected, actual);

return None;
```

```
}


// 2. 샘플링 검증
let is_corrupted = self.shared.spawner.spawn_block_in_place(|| {
let mut to_validate = BF::from_boxed_slice(
vec![0u8; self.metadata.lengths().piece_bitfield_bytes()].into_boxed_slice()
);
// 각 파일당 최소 1개 조각 검증
for fi in self.metadata.file_infos.iter() {
for piece_id in hp.as_slice()
.get(fi.piece_range_usize())
.into_iter()
.flat_map(|s| s.iter_ones())
.map(|pid| pid + fi.piece_range_usize().start)
.take(1)
{
to_validate.set(piece_id, true);
}
}
// 무작위 추가 검증
let mut queue: Vec<_> = hp.as_slice().iter_ones().collect();
queue.shuffle(&mut rand::rng());
for (idx, piece_id) in queue.into_iter().enumerate() {
```

```
let probability = 1.0 / (idx + 1).min(50) as f64;
if rand::rng().random::<f64>() < probability {
to_validate.set(piece_id, true);
}
}
// 실제 검증 수행
let fo = FileOps::new(&self.metadata.info, &self.files, &self.metadata.file_info
for piece_id in to_validate.iter_ones() {
if fo.check_piece(piece_id).is_err() {
return true; // 손상됨
}
}
false
});


if is_corrupted {
warn!("FastResume data corrupted, clearing and performing full check");
// 손상된 데이터 정리
if let Err(e) = bitv_factory.clear(self.shared.id.into()).await {
warn!("Failed to clear corrupted bitfield: {:#}", e);
}
// 전체 검사로 폴백
self.checked_bytes.store(0, Ordering::Relaxed);
```

```
return None;
}

Some(hp)
}
```

리소스 관리 및 정리

4. 메모리 관리

Arc 참조 카운팅:

```
impl Session {
// Weak 참조를 통한 순환 참조 방지
pub fn downgrade(&self) -> Weak<Session> {
Arc::downgrade(self)
}
// 백그라운드 태스크에서 Weak 참조 사용
fn spawn_background_task<F>(&self, name: &'static str, future: F)
where
F: Future<Output = anyhow::Result<()>> + Send + 'static,
{
let weak_session = Arc::downgrade(self);
self.spawn(
```

```
debug_span!(parent: self.rs(), name),
name,
async move {
// Weak 참조 업그레이드 시도
let session = match weak_session.upgrade() {
Some(s) => s,
None => {
debug!("Session dropped, terminating background task");
return Ok(());
}
};
future.await
},
);
}
}
```

4.1 파일 핸들 관리

자동 파일 정리:

```
impl Drop for MmapBitV {
fn drop(&mut self) {
// 메모리 매핑 해제 전 플러시
```

```
if let Err(e) = self.flush() {
warn!("Failed to flush bitfield during drop: {:#}", e);
}
// 메모리 매핑과 파일 핸들 자동 해제
debug!("Dropping memory-mapped bitfield");
}
}

impl Drop for FilesystemStorage {
fn drop(&mut self) {
// 모든 열린 파일 핸들 정리
for (_, file) in self.open_files.drain() {
if let Err(e) = file.sync_all() {
warn!("Failed to sync file during drop: {:#}", e);
}
}
debug!("Dropped filesystem storage, all files synced");
}
}
```

4.2 네트워크 리소스 정리

연결 정리:

```
impl Drop for PeerConnection {
fn drop(&mut self) {
debug!("Dropping peer connection to {}", self.addr);
// 진행 중인 요청들을 다른 피어로 재분배
if !self.inflight_requests.is_empty() {
warn!("Peer connection dropped with {} inflight requests",
self.inflight_requests.len());
// 요청들을 토렌트 상태로 반환
for chunk in self.inflight_requests.drain() {
self.state.requeue_chunk(chunk);
}
}
}
}
```

5. 상태 저장 메커니즘

주기적 상태 저장:

```
impl Session {
fn start_periodic_state_save(&self) {
if let Some(persistence) = &self.persistence {
let persistence = persistence.clone();
let weak_session = Arc::downgrade(self);
```

```
self.spawn(
debug_span!(parent: self.rs(), "periodic_save"),
"periodic_save",
async move {
let mut interval = tokio::time::interval(Duration::from_secs(300)); // 5분마다
loop {
interval.tick().await;
let session = match weak_session.upgrade() {
Some(s) => s,
None => break,
};
// 변경된 토렌트들만 저장
for (id, torrent) in session.db.read().torrents.iter() {
if torrent.needs_save() {
if let Err(e) = persistence.store(*id, torrent).await {
warn!("Failed to save torrent {} during periodic save: {:#}", id, e);
} else {
torrent.mark_saved();
}
}
}
}
Ok(())
```

```
},
);
}
}
}
```

5.1 비트필드 지연 플러시

메모리 효율적 플러시:

```
impl TorrentStateLive {
const FLUSH_BITV_EVERY_BYTES: u64 = 16 * 1024 * 1024; // 16MB마다 플러시
fn try_flush_bitv(&mut self) {
if self.unflushed_bitv_bytes >= Self::FLUSH_BITV_EVERY_BYTES {
if let Some(chunks) = self.chunks.as_mut() {
if let Err(e) = chunks.get_have_pieces_mut().flush() {
warn!("Failed to flush bitfield: {:#}", e);
} else {
self.unflushed_bitv_bytes = 0;
debug!("Flushed bitfield after {} bytes", Self::FLUSH_BITV_EVERY_BYTES);
}
}
}
}
```

```
fn on_piece_completed(&mut self, piece_index: ValidPieceIndex) {
// 조각 완료 시 비트필드 업데이트
if let Some(chunks) = self.chunks.as_mut() {
chunks.mark_piece_downloaded(piece_index);
// 플러시 카운터 업데이트
let piece_size = self.shared.metadata.lengths().piece_length(piece_index) as u64
self.unflushed_bitv_bytes += piece_size;
// 필요시 플러시
self.try_flush_bitv();
}
}
}
```

오류 복구 전략

6. 부분 실패 처리

토렌트별 독립적 처리:

```
impl Session {
async fn handle_torrent_error(&self, id: TorrentId, error: anyhow::Error) {
warn!("Torrent {} encountered error: {:#}", id, error);
// 1. 해당 토렌트만 오류 상태로 전환
if let Some(torrent) = self.db.read().torrents.get(&id) {
```

```
torrent.stop_with_error(error.clone());
}
// 2. 다른 토렌트들은 계속 실행
info!("Other torrents continue running despite error in torrent {}", id);
// 3. 복구 시도 스케줄링
self.schedule_recovery_attempt(id, error).await;
}
async fn schedule_recovery_attempt(&self, id: TorrentId, error: anyhow::Error) {
let weak_session = Arc::downgrade(self);
tokio::spawn(async move {
// 지수 백오프로 복구 시도
let mut delay = Duration::from_secs(30);
for attempt in 1..=3 {
tokio::time::sleep(delay).await;
let session = match weak_session.upgrade() {
Some(s) => s,
None => return,
};
info!("Attempting recovery for torrent {} (attempt {})", id, attempt);
match session.restart_torrent(id).await {
Ok(_) => {
info!("Successfully recovered torrent {}", id);
return;
```

```
}
Err(e) => {
warn!("Recovery attempt {} failed for torrent {}: {:#}", attempt, id, e);
delay *= 2; // 지수 백오프
}
}
}
error!("All recovery attempts failed for torrent {}", id);
});
}
}
```

6.1 데이터 무결성 보장

원자적 상태 변경:

```
impl ManagedTorrent {
pub fn atomic_state_change<F, R>(&self, f: F) -> anyhow::Result<R>
where
F: FnOnce(&mut ManagedTorrentLocked) -> anyhow::Result<R>,
{
let mut guard = self.locked.write();
// 상태 변경 전 백업
let backup_state = guard.state.clone();
```

```
match f(&mut *guard) {
Ok(result) => {
// 성공 시 알림
self.state_change_notify.notify_waiters();
Ok(result)
}
Err(e) => {
// 실패 시 롤백
warn!("State change failed, rolling back: {:#}", e);
guard.state = backup_state;
Err(e)
}
}
}
}
```

성능 최적화

7. 시작 시간 최적화

병렬 초기화:

```
impl Session {
async fn parallel_initialization(opts: SessionOptions) -> anyhow::Result<Arc<Sel
```

```
// 독립적인 초기화 작업들을 병렬로 수행
let (
listen_result,
dht,
(persistence, bitv_factory),
(reqwest_client, udp_tracker_client),
) = tokio::try_join!(
async {
if let Some(listen_opts) = opts.listen {
listen_opts.start(/* ... */).await.map(Some)
} else {
Ok(None)
}
},
initialize_dht(&opts),
persistence_factory(&opts),
initialize_network_clients(&opts),
)?;
// Session 생성 및 백그라운드 태스크 시작
let session = Arc::new(Session::new_with_components(
listen_result,
dht,
persistence,
```

```
bitv_factory,
reqwest_client,
udp_tracker_client,
opts,
));
// 백그라운드 태스크들도 병렬로 시작
session.start_all_background_tasks().await?;
Ok(session)
}
}
```

7.1 종료 시간 최적화

타임아웃 기반 강제 종료:

```
impl Session {
pub async fn shutdown_with_timeout(&self, timeout: Duration) -> anyhow::Result<(
info!("Starting graceful shutdown with {}s timeout", timeout.as_secs());
// 1. 취소 신호 전송
self.cancellation_token.cancel();
// 2. 타임아웃과 함께 정리 작업 수행
match tokio::time::timeout(timeout, self.graceful_shutdown()).await {
Ok(result) => {
info!("Graceful shutdown completed");
```

```
result
}
Err(_) => {
warn!("Graceful shutdown timed out, forcing termination");
self.force_shutdown().await
}
}
}
async fn graceful_shutdown(&self) -> anyhow::Result<()> {
// 1. 모든 토렌트 일시정지
let shutdown_futures: Vec<_> = self.db.read()
.torrents
.values()
.map(|torrent| torrent.shutdown())
.collect();
futures::future::try_join_all(shutdown_futures).await?;
// 2. DHT 종료
if let Some(dht) = &self.dht {
dht.shutdown().await?;
}
// 3. 지속성 저장소 플러시
if let Some(persistence) = &self.persistence {
persistence.shutdown().await?;
```

```
}
Ok(())
}
async fn force_shutdown(&self) -> anyhow::Result<()> {
warn!("Performing forced shutdown");
// 긴급 상태 저장만 수행
if let Some(persistence) = &self.persistence {
for (id, torrent) in self.db.read().torrents.iter() {
let _ = persistence.store(*id, torrent).await;
}
}
Ok(())
}
}
```

결론

rqbit의 시스템 시작 및 종료 과정은 다음과 같은 특징을 가집니다:

시작 과정의 특징:

1. **체계적인 초기화 순서**: 의존성을 고려한 단계별 초기화
2. **병렬 처리**: 독립적인 구성 요소들의 동시 초기화
3. **견고한 복원**: FastResume과 검증을 통한 안전한 상태 복원

4. **오류 격리**: 일부 구성 요소 실패가 전체에 영향 주지 않음

종료 과정의 특징:

1. **우아한 종료**: 취소 토큰을 통한 계층적 정리
2. **데이터 보존**: 모든 중요 상태의 안전한 저장
3. **타임아웃 보호**: 무한 대기 방지를 위한 강제 종료
4. **리소스 정리**: 메모리, 파일, 네트워크 리소스의 완전한 해제

복구 메커니즘:

1. **부분 실패 처리**: 개별 토렌트 오류가 전체에 영향 없음
2. **자동 복구**: 지수 백오프를 통한 자동 재시도
3. **데이터 무결성**: 원자적 상태 변경과 롤백
4. **긴급 저장**: 패닉 상황에서도 중요 데이터 보존

이러한 설계를 통해 rqbit은 안정적이고 신뢰할 수 있는 시스템 생명주기 관리를 제공하며, 다양한 오류 상황에서도 데이터 손실 없이 안전하게 동작합니다.